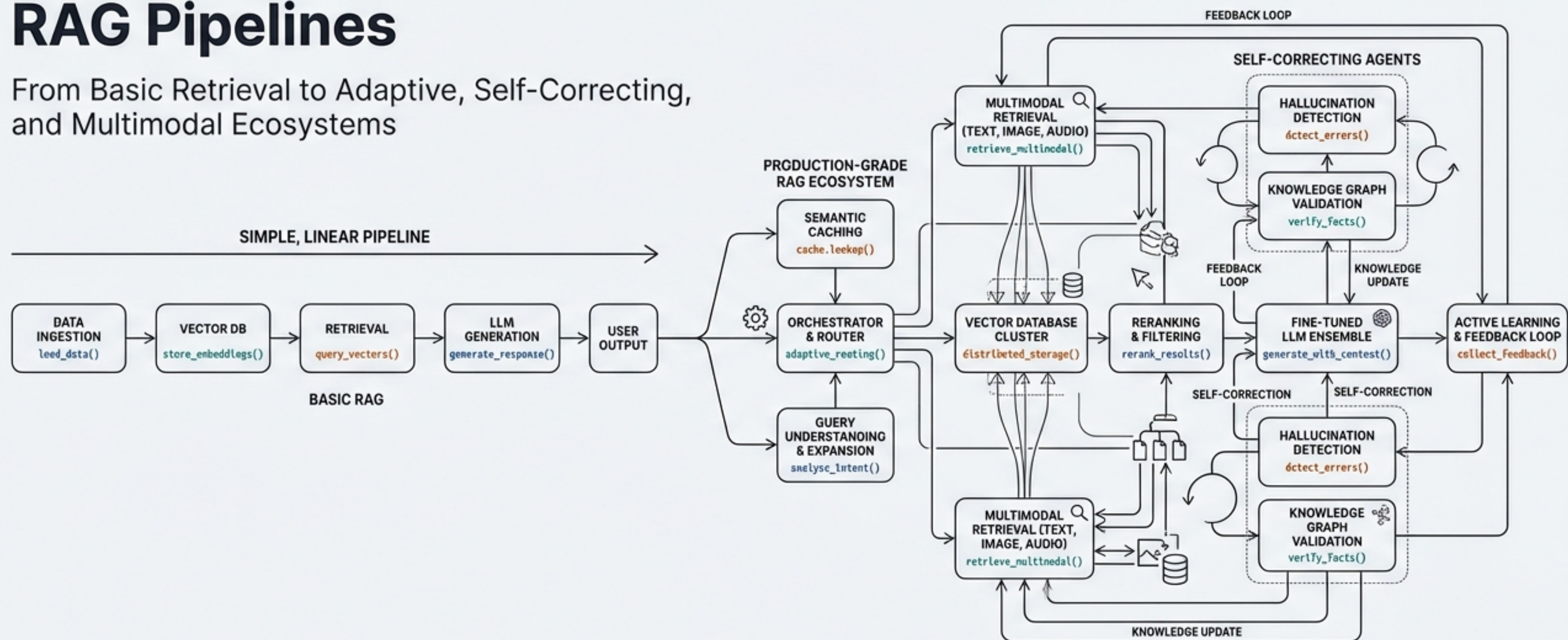


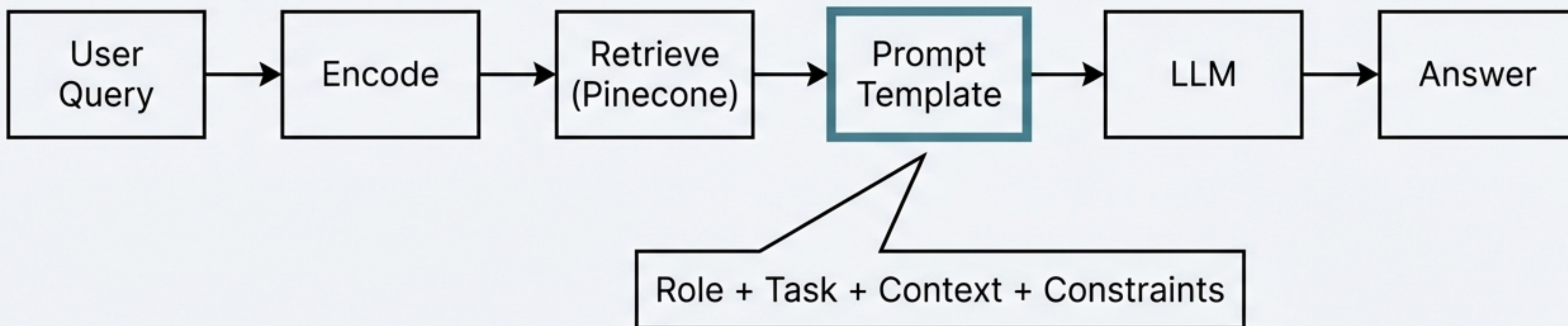
Building Production-Grade RAG Pipelines

From Basic Retrieval to Adaptive, Self-Correcting, and Multimodal Ecosystems



Zero to Hero: The Evolution of Intelligent Agents.

The Foundation: Transforming Indices into Assistants

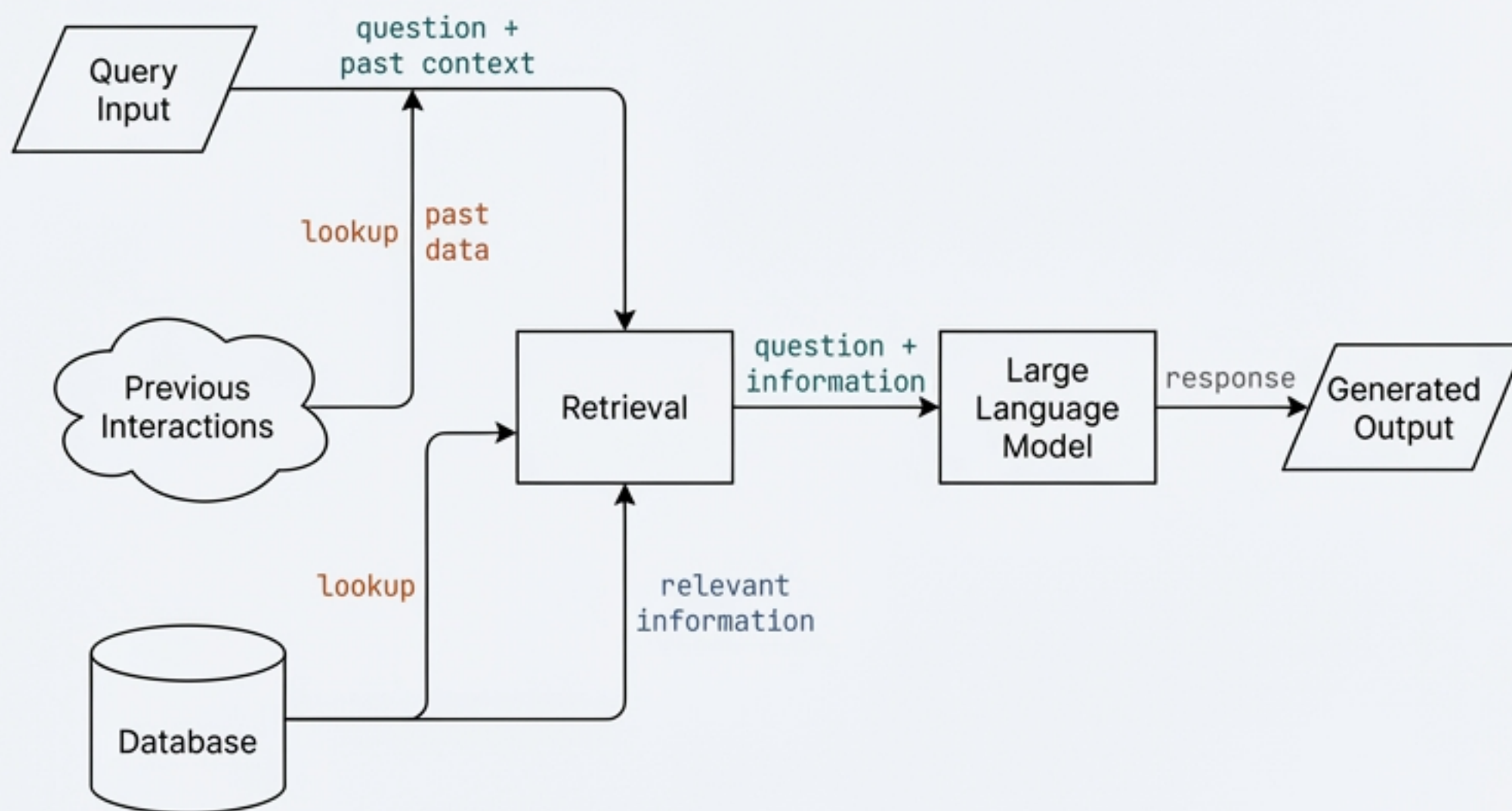


```
search_results = index.query(vector=embedding, top_k=5)
context = "\n".join([match['metadata']['text'] for match in search_results])
prompt = template.format(context=context, query=user_query)
```

Insight: Basic RAG relies on a 'Single Shot' architecture—one chance to retrieve, one chance to answer.

Adding Continuity: Memory-Based RAG

- The Problem & Solution
- Basic RAG is stateless; it forgets previous turns.
- Solution: Dual Retrieval.
 1. Short-term: Rolling chat history.
 2. Long-term: User facts stored in vector namespace.

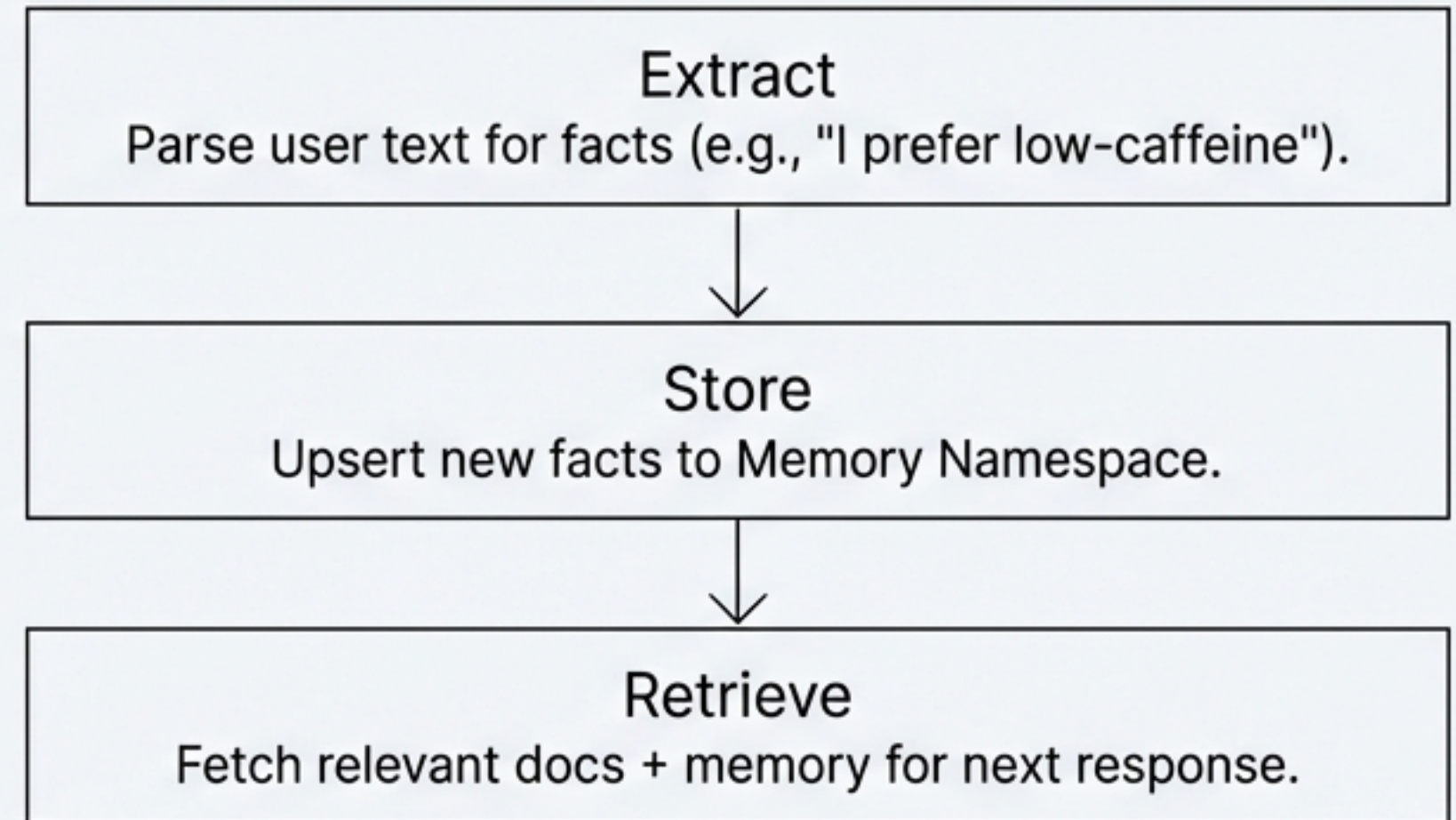


Implementation: The Extract-Store-Retrieve Cycle

The Template (YAML)

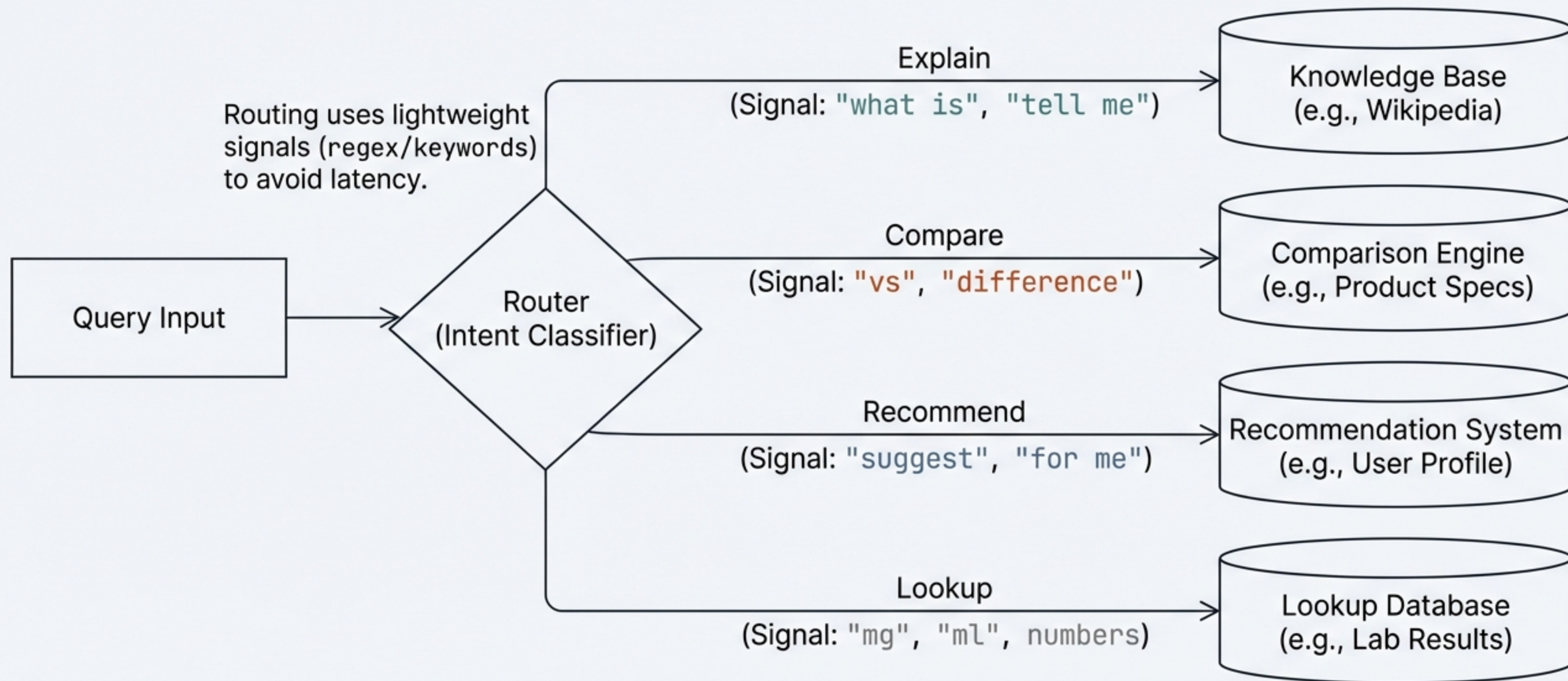
```
1  template: |
2    Conversation so far: {history}
3    Relevant memory: {memory_block}
4    Context: {context_block}
5    User: {query}
6
```

The Turn Cycle

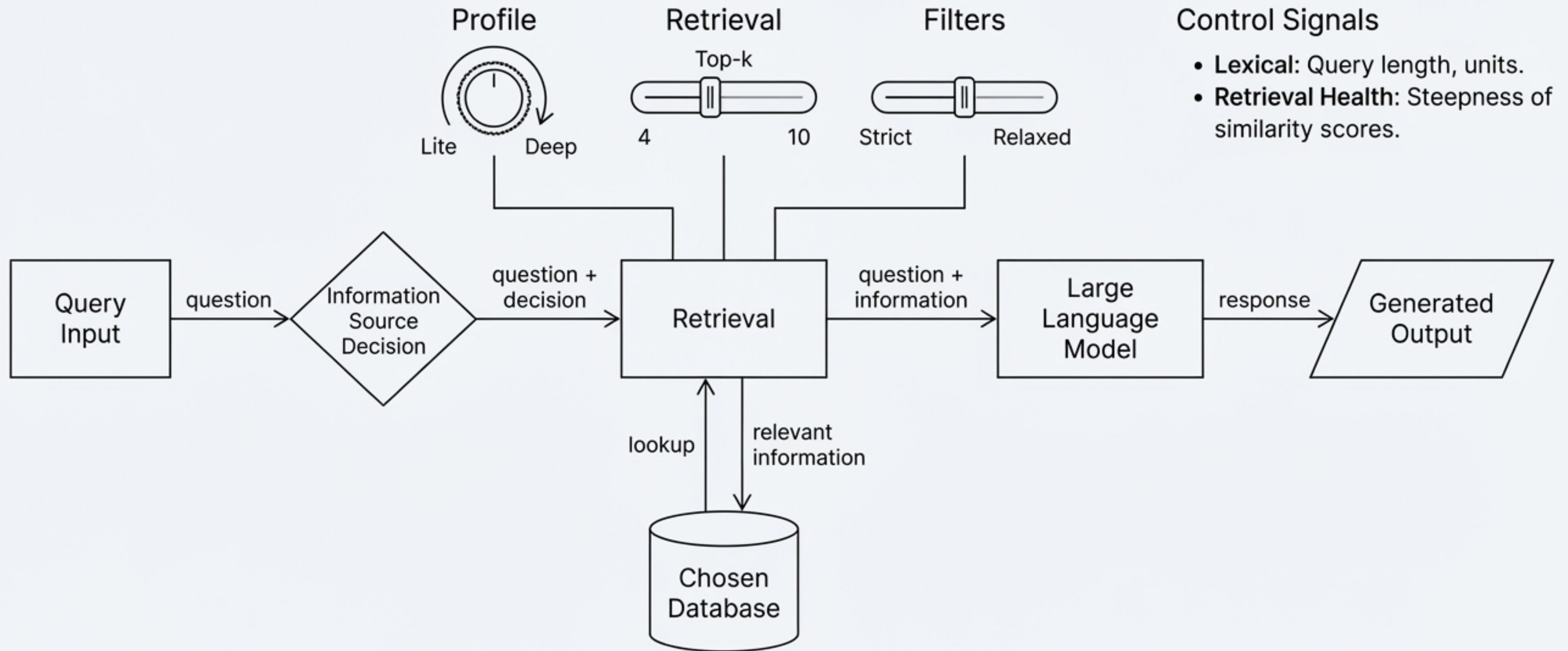


Critical: Explicit role-labeling ("User" vs "Assistant") prevents the model from confusing past outputs with new instructions.

Routing by Intent: Branched RAG



Dynamic Tuning: Adaptive RAG



Adaptive Logic in Action

Scenario A (Lite Profile)

Input

Caffeine in Arabica?

Signal

Units detected.

Action

profile='lite'
k=4

Scenario B (Deep Profile)

Input

Compare flavor profiles...

Signal

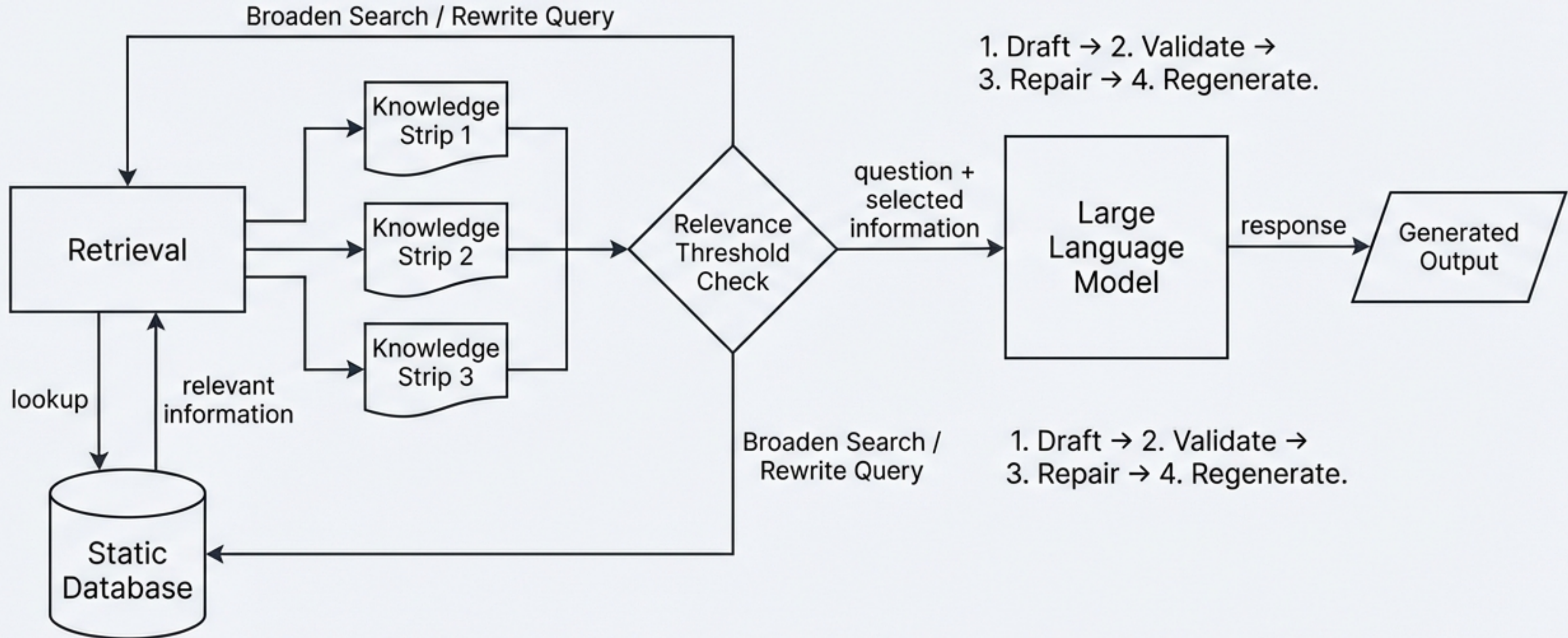
Comparison detected.

Action

profile='deep'
k=10

```
if sig["has_units"] or prb["steep"] > 0.05:  
    return {"profile": "lite", "k": 4}  
elif sig["is_compare"]:  
    return {"profile": "deep", "k": 10}
```


The Repair Loop: Corrective RAG



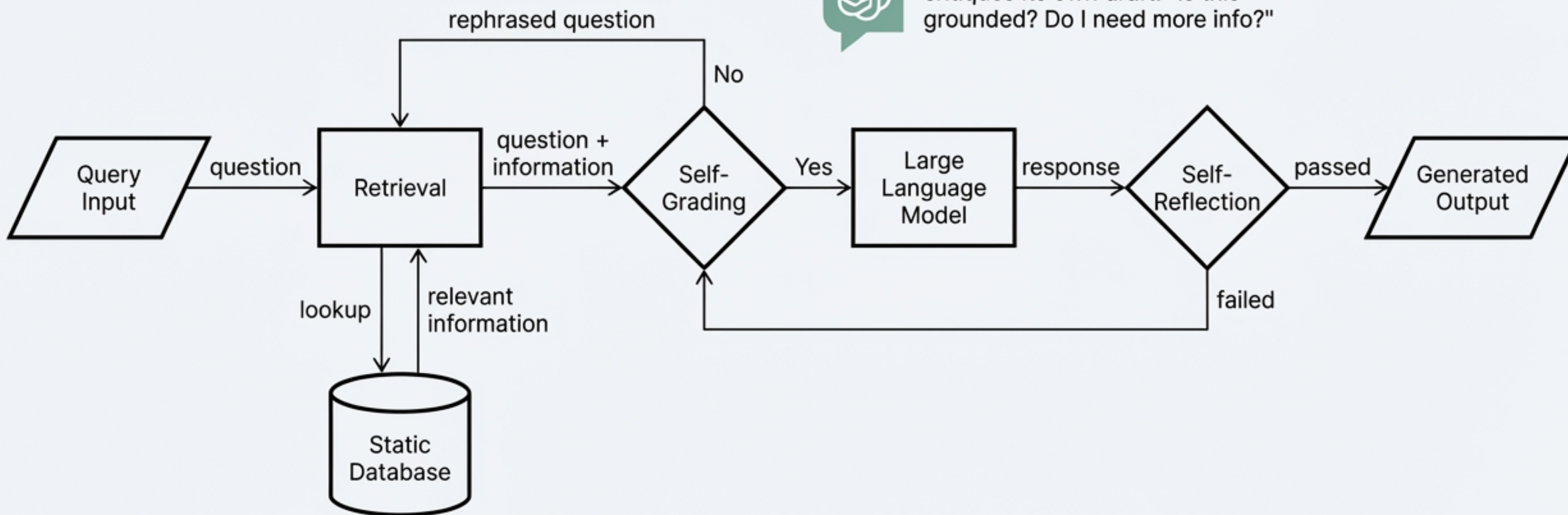
Corrective Mechanics & Metrics

The Validators (Metrics)			
E Evidence Adequacy: Do we have enough data?	F Faithfulness: Is the answer grounded in docs?	C Coverage: Did we answer the whole question?	R Rules: Format compliance (e.g., citations).
Repair Tactics			
Query Rewriting: "Clarify entities ("kesar" → "saffron")." Retriever Switching: Swap Dense for Lexical (for numbers/IDs). Targeted Re-retrieval: Isolate ungrounded claims and search again.			

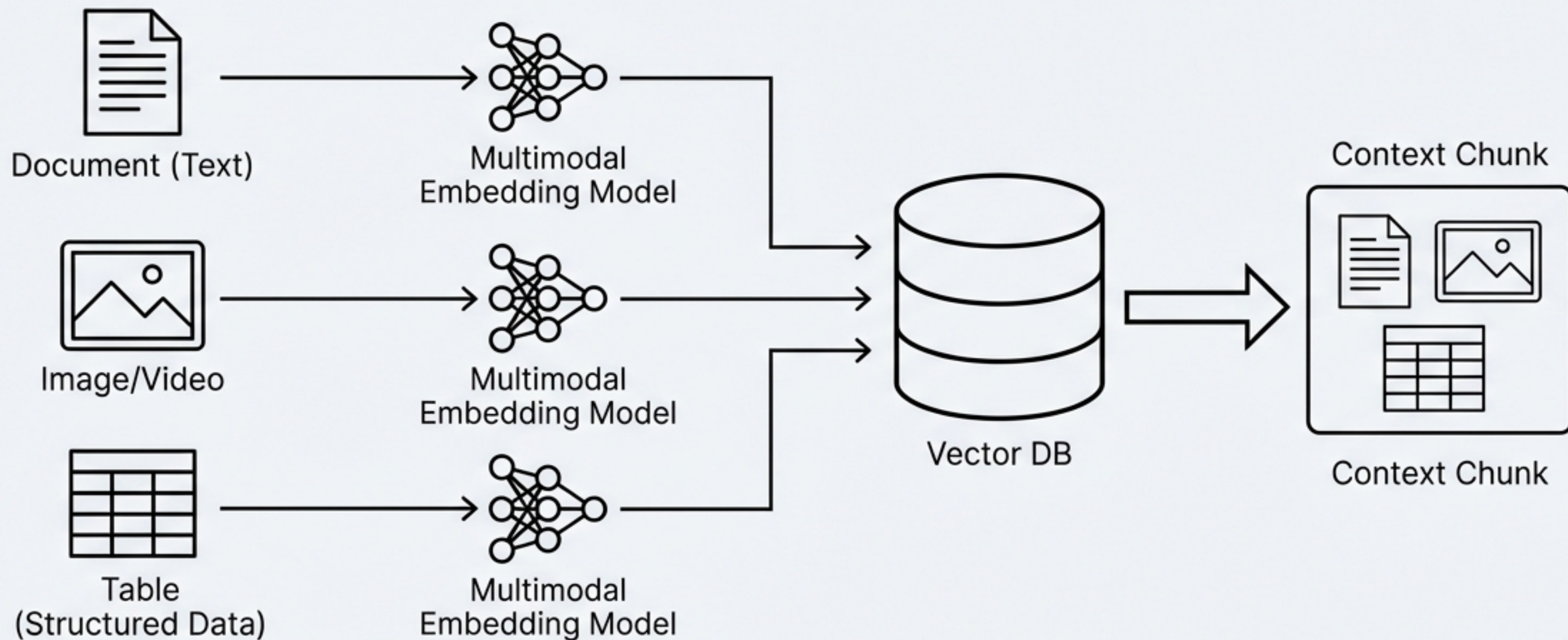
Introspection: Self-RAG



Internal Monologue: The model critiques its own draft. "Is this grounded? Do I need more info?"



Expanding Senses: Multimodal RAG



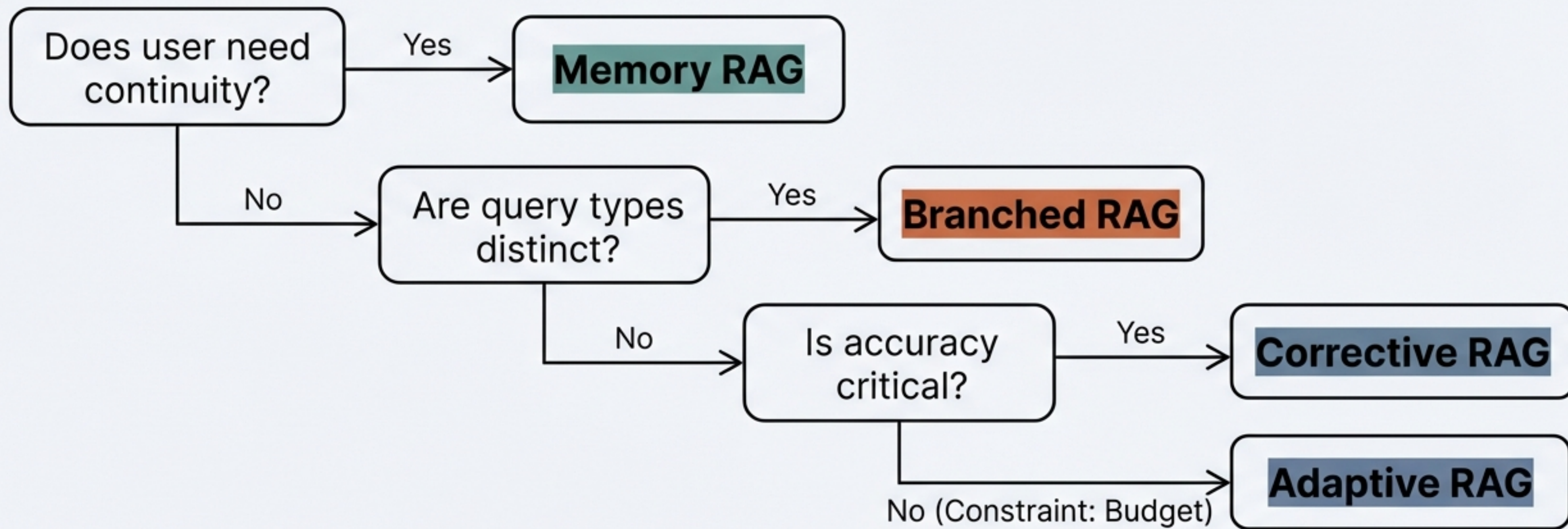
Unified Store: Shared embedding space (e.g., CLIP) allows cross-modal retrieval (Text query → Image result).

Architecture Comparison Matrix

Strategy	Best Use Case	Trade-off
Memory RAG	Personalization	Context overhead
Branched RAG	Heterogeneous Intents	Routing complexity
Adaptive RAG	Cost/Latency Balance	Tuning heuristics
Corrective RAG	High Compliance/Stakes	High Latency (loops)
Self-RAG	Complex Reasoning	High Token cost

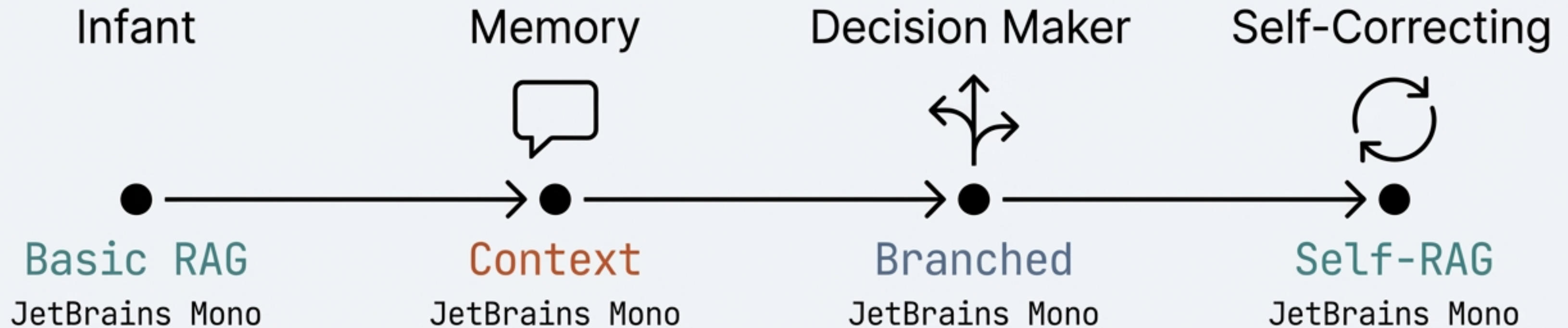
“The art is in choosing the right variant for the right problem.”

Choosing Your Strategy



Production systems often chain these: **Branched RAG** (Intent) → **Adaptive RAG** (Knobs) → **Corrective RAG** (Safety Net).

The Future: Self-Improving Ecosystems



Move beyond static prompting. Design agents that can adapt, repair, and reflect to survive in the wild.